

POD-Based Model of Tollmien–Schlichting Waves

SARI Ayoub, JORON Joas.

July 29, 2026

Abstract

Tollmien-Schlichting (TS) waves are a type of instability that can occur in the boundary layers of fluid flows. They are characterized by low amplitude oscillations that grow exponentially with time and can eventually lead to turbulence.

The POD-based model presented in this project is a simplification contribution to the study of TS waves. By reducing the complexity of the original system through the use of the POD method, we provide a more efficient and simplified way to analyze and predict the behavior of fluid flows under blowing and suction. Additionally, the mathematical techniques employed in this project, such as SVD and ROM, have many applications in engineering, applied science, and in processing large datasets.

Overall, this project aims to apply and understand the mathematical methods studied in the master's program and to demonstrate the usefulness and effectiveness of POD-based models in reducing the complexity of dynamical systems. The results presented in this article have implications for a wide range of applications in fluid mechanics and beyond.

INTRODUCTION

Laminar flows represent the situations where a fluid flows in a particular direction, without displaying differences at a local scale. This regime allows to define a streamwise direction and a cross-stream direction and can describe many natural situations and engineering configurations, where a strong shear in the cross-stream directions develops from a specific point such as a solid obstacle. This article deals two-dimensional boundary layer flows, i.e. situations where laminar flow develops along a semi-infinite planar plate. This fundamental problem is of course very important in aeronautics and hydrodynamics. It is found that the pressure is uniform in the boundary layer, and the velocity profile at the wall, called the Blasius profile, can be determined by simple numerical calculations.

The interest shown in this subject comes from the possible instability of this flow. In this context, one can distinguish flows that behave as oscillators and flows which act as noise-amplifiers. In this work, flow control techniques will be evaluated using a test case of the latter. In which perturbation have a broad range of frequencies and only exist if the flow is perturbed by an external noise source.

Basically, the control takes as input one or many measurements of the system and returns the temporal behavior of the actuator, namely the control law. In flow control applications the cost functional of the control gain matrix is usually taken as the sum of the state energy (or the measurement energy) and the energy of the control.

In addition, to compute a control law in real time, the compensator (controller and estimator) should have a reaction time similar to the phenomena to be controlled. This leads to a reduced-order model.

Two major strategies are available to arrive at a reduced-order model: system identification techniques and projection techniques. For the latter, The starting point is : The starting point of this method is :

- (i) a high-dimensional state-space model arising from the discretization of the linearized Navier-Stokes equations with actuators and sensors
- (ii) a set of vectors, called the reduced basis, which span the subspace onto which the dynamics will be projected.

The high-dimensional system is then projected onto the reduced basis using a Galerkin projection. This results in a reduced-order model whose size is equal to the dimension of the projection basis.

The article is organized as follows: we will study the configuration of the boundary layer flow and point out that the dynamics are governed by linearized Navier-Stokes equations. A lifting procedure will be conducted in a localized region i.e. a normal velocity profile will be imposed at the wall to obtain the state-space formulation. Simulations are run with Freefem++ programs and post-processing is conducted with Python codes. We will evaluate the impulse response of the configuration, generate POD modes, and perform model order reduction by the Galerkin projection.

I. CONFIGURATION AND MATHEMATICAL BACKGROUND

A. Configuration

We consider a two-dimensional configuration for this problem. In a uniform flow of velocity U_∞ . At the initial time, the flow field is equal to zero thus we set $\mathbf{u}(t=0) = (0,0)$ on the inlet

The base flow $\mathbf{u}_0 = (u_0, v_0)$ where the fluid enters the domain from the left of the domain (figure 7) where a uniform streamwise velocity is prescribed.

$$\mathbf{u}_0 = (1, 0)$$

While a Dirichlet no-slip boundary condition is set on the wall boundary. This will generate a boundary layer of moderate thickness.

$$\mathbf{u}_{\text{wall}} = (0, 0)$$

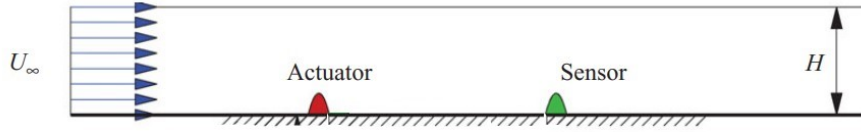


Figure 1: Sketch of the geometry of the flow

The Reynolds number used to characterize the different flows we study is given by :

$$Re = \frac{U_\infty H}{\nu}$$

where ν is the fluid kinematic viscosity, h the channel height.

The fluid motion is described by the two-dimensional unsteady incompressible Navier-Stokes equation:

$$\begin{aligned} \partial_t \mathbf{u} + (\mathbf{u} \cdot \nabla) \mathbf{u} &= -\nabla P + \frac{1}{Re} \Delta \mathbf{u} \\ \nabla \cdot \mathbf{u} &= 0 \end{aligned} \quad (1)$$

The governing equation 1 can be recast into:

$$\mathcal{B} \partial_t \mathbf{w} + \frac{1}{2} \mathcal{N}(\mathbf{w}, \mathbf{w}) + \mathcal{L} \mathbf{w} = 0 \quad (2)$$

Where $\mathbf{u} = \begin{pmatrix} u \\ v \end{pmatrix}$ and $\mathbf{w} = \begin{pmatrix} \mathbf{u} \\ P \end{pmatrix}$. By introducing the following operators:

- A prolongation operator $\mathcal{B}$ that transforms $\mathbf{w}$ into $\begin{pmatrix} \mathbf{u} \\ 0 \end{pmatrix}$

$$\mathcal{B} = \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix}$$

- A bi-linear operator $\mathcal{N}$ that accounts for the flow advection:

$$\mathcal{N}(\mathbf{w}_1, \mathbf{w}_2) = \begin{pmatrix} \mathbf{u}_1 \cdot \nabla \mathbf{u}_2 + \mathbf{u}_2 \cdot \nabla \mathbf{u}_1 \\ 0 \end{pmatrix}$$

- A linear operator (Stokes):

$$\mathcal{L} = \begin{pmatrix} -Re^{-1} \Delta() & \nabla() \\ -\nabla() & 0 \end{pmatrix}$$

By decomposing the solution into a base flow w_0 and a perturbation w_1 such as : $\mathbf{w} = w_0 + \epsilon w_1$ with $\epsilon \ll 1$, we get:

$$\mathcal{B} \partial_t (\mathbf{w}_0 + \epsilon \mathbf{w}_1) + \frac{1}{2} \mathcal{N}(\mathbf{w}_0 + \epsilon \mathbf{w}_1, \mathbf{w}_0 + \epsilon \mathbf{w}_1) + \mathcal{L}(\mathbf{w}_0 + \epsilon \mathbf{w}_1) = 0$$

- Order $O(1)$: $\mathcal{B}\partial_t \mathbf{w}_0 + \frac{1}{2}\mathcal{N}(\mathbf{w}_0, \mathbf{w}_0) + \mathcal{L}\mathbf{w}_0 = 0$
- Order $O(\epsilon)$: $\mathcal{B}\partial_t \mathbf{w}_1 + \underbrace{\frac{1}{2}[\mathcal{N}(\mathbf{w}_1, \mathbf{w}_0) + \mathcal{N}(\mathbf{w}_0, \mathbf{w}_1)]}_{\mathcal{N}_{w_0} \mathbf{w}_1} + \mathcal{L}\mathbf{w}_1 = 0$

At order 0, we remove the time derivative because the base-flow w_0 is considered time independent. The linear operator $\mathcal{N}$ under the curly bracket is given by:

$$\mathcal{N}_{w_0} \mathbf{w}_1 = \begin{pmatrix} \mathbf{u}_1 \cdot \nabla \mathbf{u}_0 + \mathbf{u}_0 \cdot \nabla \mathbf{u}_1 \\ 0 \end{pmatrix}$$

B. Governing Equations

The linearized governing equations can be written in the form of an initial-value problem according to:

$$\mathcal{B} \frac{d}{dt} \mathbf{w}(x, y, t) + \mathcal{L}_{w_0} \mathbf{w}(x, y, t) = 0 \quad (3)$$

We define the linearized Navier-Stokes operator as : $\mathcal{L}_{w_0} = \begin{pmatrix} \nabla() \cdot \mathbf{u}_0 + \nabla \mathbf{u}_0 \cdot () - Re^{-1} \Delta() & -\nabla() \\ \nabla \cdot () & 0 \end{pmatrix}$

A transformation, referred to as lifting, is used to render the control problem homogeneous.

We decompose the state vector $\mathbf{w}$ into: a solution to the homogeneous problem $\mathbf{w}'$ (with no control applied $u(t) = 0$) and a solution $\mathbf{w}_f$ of the steady but inhomogeneous problem (with the specific constant control $u(t) = 1$ applied) such that :

$$\mathbf{w} = \mathbf{w}' + \mathbf{w}_f u(t)$$

$$s = -ikc = -ikc_r + \underbrace{kc_i}_{\text{growth rate}}$$

$$s = s_r + is_i$$

$$c = c_r + ic_i$$

$$0 < kd < k_s d \quad \underbrace{\hspace{1cm}}_{\text{coupling term}}$$

$$kd > k_s d$$

$$\mathcal{I}_m(c) = c_i > 0 \quad (4)$$

upon substitution into the governing equation 3 we obtain:

$$B\dot{w}' + \mathcal{L}_{w_0} w' + \dot{u}Bw_f + u\mathcal{L}_{w_0} w_f = 0 \quad (5)$$

Note that $u(t)$ is an input that determines the magnitude and temporal behavior of w_f . At the border $w_f = f(x)$ and $w' = 0$.

Given that :

$$\mathcal{L}_{w_0} w_f = 0 \quad (6)$$

We assume a localized region Ω_C of the wall i.e. the actuator's location to impose a weak forcing. By associating a control function $f(x, y)$ at the wall such that: $w_f = f(x)$, which yields :

$$u_f(y = 0) = 0$$

$$v_f(y = 0) = f(x)u(t)$$

hence, a normal velocity profile is applied at the actuator location Ω_C .

The wall-normal shear stress was integrated over a localized region Ω_M of the wall i.e. the sensor's location. We obtain:

$$m = \int_{\Omega_M} \frac{\partial u}{\partial y} \Big|_{y=0} dx$$

We formulate the continuous time-invariant state-space model for our flow control problem:

$$\begin{cases} \mathcal{B}\dot{w}'(t) = -\mathcal{L}_{w_0}w'(t) + \mathcal{B}w_f c(t) \\ m(t) = Mw'(t) \end{cases}$$

Where: $w'(t)$, $c(t) = -\frac{du}{dt}$, $m(t)$ are respectively: the state, the control law (input), and the measurement (output).

After meshing the domain, the discretized form of the governing equations 5 can be written :

$$\begin{cases} Q\dot{X} = AX + QCc \\ m = MX \end{cases} \quad (7)$$

$$AC = 0 \quad (8)$$

By defining : $X = w'$ $Q = \mathcal{B}$ $A = -\mathcal{L}_{w_0}$ $C = w_f$

Assuming n degrees of freedom for the state vector X . The above matrices are of the following size: $Q, A \in \mathbb{R}^{n \times n}$, $C \in \mathbb{R}^{n \times 1}$, $M \in \mathbb{R}^{1 \times n}$. This explicit discrete system can be transformed into a reduced-order model using a general projection basis by the Galerkin projection.

C. Model Reduction

We start by introducing a general basis given by V and its dual equivalent V^* of the form :

$$\underbrace{V^*QV}_{\text{bi-orthogonality}} = I$$

Where: I stands for the identity matrix.

The discretized state vector X is then expressed as a linear combination of the columns of the chosen expansion basis V according to:

$$X = V\hat{X}, \quad \hat{X} = V^*QX$$

A straightforward Petrov–Galerkin projection then yields the reduced system :

$$\begin{cases} \hat{X} = \hat{A}\hat{X} + \hat{C}c \\ m = \hat{M}\hat{X} \end{cases} \quad (9)$$

With the reduced system, control, and measurement matrices are given as:

$$\hat{A} = V^*AV \quad \hat{C} = V^*QC \quad \hat{M} = mV$$

Depending on the bases used for the model reduction, the reduced system should be able to represent critical features of the full system. In our study, we chose the reduction based on proper orthogonal decomposition (POD) modes.

D. Proper Orthogonal Decomposition

1) *Generate POD on snapshot w'* : Generating data such as snapshots of w' requires a large data storage and technical processing capacity of modeling. The analysis of the unsteady and non-linear flows as well as the large amount of data requires a reduced order modeling. It is done here by the Proper Orthogonal Decomposition (POD). POD is a method for identifying a set of dominant patterns in a data set, such as a time series of measurements or simulations. Thus, we can draw the main information from our flow here. The matrix X containing N snapshots, where each column represents a single snapshot at a particular time. The POD basis is a set of orthonormal basis vectors that cover the dominant modes or patterns of :

$$X_{ij} = w'(x_i, t_j) \quad (10)$$

We assume that we have

- N snapshots of dimension n ($N \ll n$):

$$X \in \mathbb{R}^{n \times N}$$

- A scalar-product in space $Q_x \in \mathbb{R}^{n \times n}$, such that $Q_x^* = Q_x : Q_x = M_x^* M_x = x_1^* Q_x x_2$

- A scalar-product in time $Q_t \in \mathbb{R}^{N \times N}$, such that $Q_t^* = Q_t$.

To obtain the POD basis, we first compute the singular value decomposition (SVD). The Singular Value Decomposition (SVD) is a factorization method for matrices that decomposes a matrix into three matrices, to extract an orthonormal matrix.

$$X = V\Lambda T^* \quad (11)$$

with

$$V^* Q_x V = T^* T$$

Here $V \in \mathbb{R}^{n \times N}$ is the orthonormal basis with respect to Q_x , V is called *TOPOS* and are the POD modes. $\Lambda \in \mathbb{R}^{N \times N}$ is an $N \times N$ diagonal matrix with non negative singular values. And $T \in \mathbb{R}^{N \times N}$ is an orthonormal basis with respect to $Q_t = I$, T is called *CHRONOS*.

Firstly we compute the chronos T , $X^* Q_x X t_i = \lambda_i^2 t_i$ and $X^* Q_x X t_j = \lambda_j^2$

$$\begin{aligned} &\Rightarrow t^* X^* Q_x X t_j = \lambda_i^2 t_i^* t_j \\ &\iff (X^* Q_x X t_i) * t_j = \lambda_i^2 t_i^* t_j \\ &\iff (\lambda_i^2 - \lambda_j^2) t_i^* t_j = \lambda_i^2 t_i^* t_j \end{aligned}$$

the eigenvectors T form an orthonormal basis with respect to $Q_t = I$ and $T^* T = I$.

We solve this equation to compute the chronos:

$$X = V\Lambda T^* \quad (12)$$

With the temporal correlation matrix defined like:

$$X^* Q_x X = \begin{bmatrix} X_1^* \\ X_2^* \\ X_3^* \\ \vdots \\ X_{n_s}^* \end{bmatrix} Q_x [X_1 X_2 X_3 \cdots X_{n_s}] = \begin{bmatrix} X_1^* Q_x X_1 & X_1^* Q_x X_2 & X_1^* Q_x X_3 & \cdots & X_1^* Q_x X_{n_s} \\ X_2^* Q_x X_1 & X_2^* Q_x X_2 & X_2^* Q_x X_3 & \cdots & X_2^* Q_x X_{n_s} \\ X_3^* Q_x X_1 & X_3^* Q_x X_2 & X_3^* Q_x X_3 & \cdots & X_3^* Q_x X_{n_s} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ X_{n_s}^* Q_x X_1 & X_{n_s}^* Q_x X_2 & X_{n_s}^* Q_x X_3 & \cdots & X_{n_s}^* Q_x X_{n_s} \end{bmatrix}$$

If $Q_x = I$ and $Y = X^*$, we have the two-time (single point) auto-correlation matrix:

$$\frac{X^* X}{n} = \frac{Y Y^*}{n} = \frac{1}{n} \sum_{i=1}^n Y_i Y_i^* = \langle Y(x) Y(x)^* \rangle_x \sim R(t_1, t_2) = \frac{1}{\Omega} \iiint_{\Omega} \phi(x, t_1) \phi(x, t_2) dx \quad (13)$$

After solved the topos, we then deduce the topos from:

$$\begin{aligned} V^* Q_x V &= \Lambda^{-1} T^* X^* Q_x X T \Lambda^{-1} \\ &\iff \Lambda^{-1} T^* T \Lambda^2 \Lambda^{-1} = I \end{aligned}$$

We solve:

$$V = X T \Lambda^{-1} \quad (14)$$

With the spatial correlation matrix defined like

$$X X^* = Y^* Y \in \mathbb{R}^{n \times n}, \quad Y = X^*$$

$$Y^* Y_0 = \begin{bmatrix} Y_1^* \\ Y_2^* \\ Y_3^* \\ \vdots \\ Y_n^* \end{bmatrix} [Y_1 Y_2 Y_3 \cdots Y_n] = \begin{bmatrix} Y_1^* Y_1 & Y_1^* Y_2 & Y_1^* Y_3 & \cdots & Y_1^* Y_n \\ Y_2^* Y_1 & Y_2^* Y_2 & Y_2^* Y_3 & \cdots & Y_2^* Y_n \\ Y_3^* Y_1 & Y_3^* Y_2 & Y_3^* Y_3 & \cdots & Y_3^* Y_n \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ Y_n^* Y_1 & Y_n^* Y_2 & Y_n^* Y_3 & \cdots & Y_n^* Y_n \end{bmatrix}$$

Here $Q_t = I$, we have the two-point (single time) auto-correlation matrix:

$$\frac{X X^*}{n_s} = \frac{1}{n_s} \sum_{i=1}^{n_s} X_i X_i^* = \langle X(t) X(t)^* \rangle_t \sim R(x_1, x_2) = \frac{1}{\Delta} \int_0^{\Delta} w'(x_1, t) w'(x_2, t) dt \quad (15)$$

2) *Reduced POD base*: The POD base is then used to project new data onto a lower dimensional space covered by the dominant modes, thus enabling dimensionality reduction and efficient data representation. By using the POD basis for flow representation, we can capture the most significant features of the data using a small number of basis vectors. This can lead to a more efficient representation of the flow, as we can represent the data using fewer parameters without losing important information. By minimizing the lower modes we also minimize the size of the data. For this, we assume that:

$$\lambda_1 > \lambda_2 > \dots > \lambda_p \gg \lambda_{p+1} > \dots$$

Consequently:

$$X = V \Lambda T^* \approx V_p \Lambda_p T_p^* \quad (16)$$

- $V_p \in \mathbb{R}^{n \times p}$, the first p topos
- $\Lambda_p \in \mathbb{R}^{p \times p}$, the first p singular values
- $T_p \in \mathbb{R}^{p \times p}$, the crhonos of th first p POD modes

This allows for more efficient data representation, as it can be represented with fewer parameters without losing important information. Therefore, it can be used to project new data into the low-dimensional space covered by the dominant modes. This means that the new data can be represented as a linear combination of POD basis vectors using the coefficients obtained by projecting the new data onto the POD basis. For example, Galerkin model we will have in expanded form:

$$\hat{\dot{X}}_p = \sum_l \hat{A}_{pl} \hat{X}_l + \sum_l \hat{C}_{pl} \hat{c} \quad (17)$$

In the following, we perform the DNS simulation of our experiments with FreeFem++ and compute the POD and Galerking projection with Python.

II. NUMERICAL STUDY

A. Input–Output Behaviour

By definition, the input–output behavior links the effect of the actuator on the flow to the information extracted by the sensor. It can be described by the impulse response or, equivalently, by the transfer function. The impulse response, i.e. the reaction of the system to a Dirac function for the control law $c(t) = \delta(t)$, produces the measurement $m(t) = MX(t)$ where X is the solution of the initial-value problem :

$$\begin{cases} Q\dot{X} = AX \\ X(t=0) = C \end{cases} \quad (18)$$

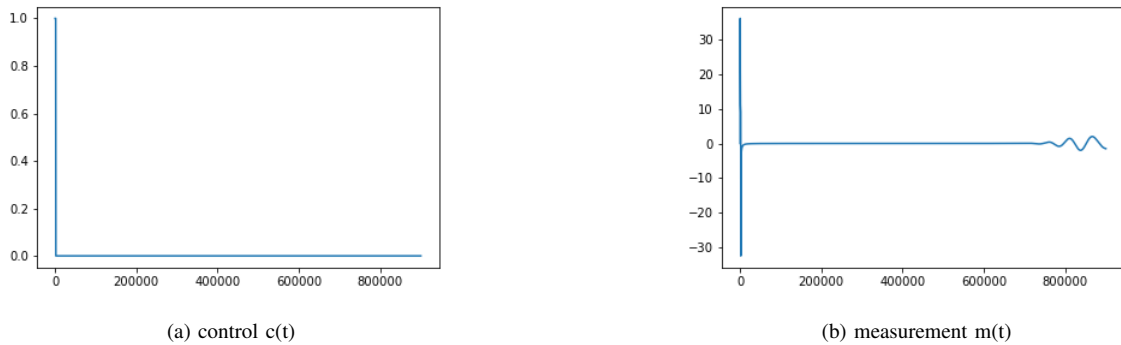


Figure 2: Impulse response as a function of time

In figure 2b, this impulse response is displayed. One can observe an immediate effect of the actuation at the sensor location, even though the sensor is far from the actuator location. After a rapid decay, the signal is completely attenuated. It tends to be slightly periodic at the end of the simulation.

B. POD results

After the calculation of the DNS with FreeFem++. On the basis of the theory we compute de POD with the python code (in appendix,). We choose $Q = Q_x$ as kinetic energy and $Q_t = I$. We form the temporal correlation matrix: $C = X^{bullet} Q X$. Solve the eigenvalue problem to compute the chronos: T

$$[T, \Lambda^2] = \text{eig}(C)$$

Since $Q_t = I$:

$$[T, \Lambda^2, \sim] = \text{svd}(C)$$

We first choose $p = 100$ modes of POD to calculate: p .

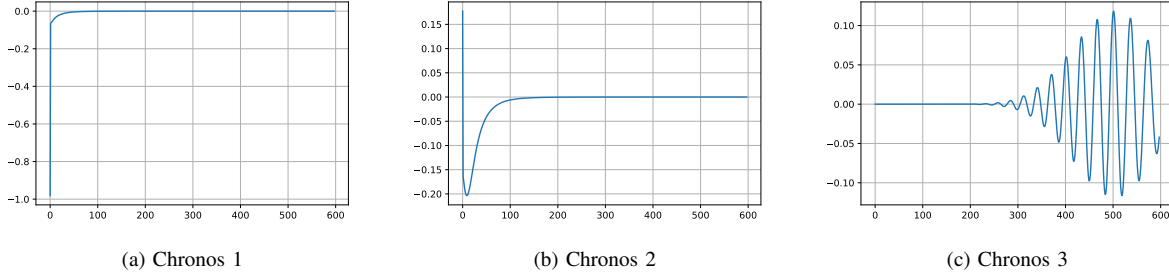


Figure 3: Firts Chronos as a function of time

We restrict T to the first p columns: T_p

And restrict Λ to the upper left block (p, p) of Λ : Λ_p We compute the topas p , the POD modes with: V

$$V = X T_p \Lambda_p^{-1}$$

The POD basis is orthogonal:

$$V^* Q V = I$$

We find an amplitude of 10^4 for the first mode and we see the amplitude of the other mode decrease almost logarithmically,

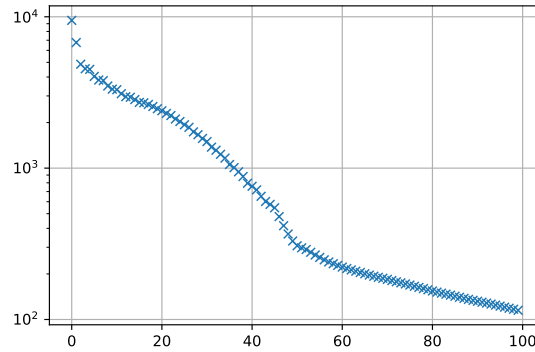


Figure 4: POD Modes for $p = 100$

as we assume in theory.

$$\lambda_1 > \lambda_2 > \dots > \lambda_p \gg \lambda_{p+1} > \dots$$

The amplitudes become very small for large p modes. We can choose a smaller number p while keeping the main information of the flow.

We plot the mode of our simulation, we see exactly the impulsion on the wall. After conducting a POD analysis on a Navier-Stokes flow with an added impulse using Free Fem, we were able to trace the simulation mode and observe the impulse's effect on the lower wall. This indicates that our POD analysis was successful and effectively captured the main features of the flow. As a result, we can conclude that our computation was accurate and reliable. However, further analysis and validation may be necessary to fully verify the results.

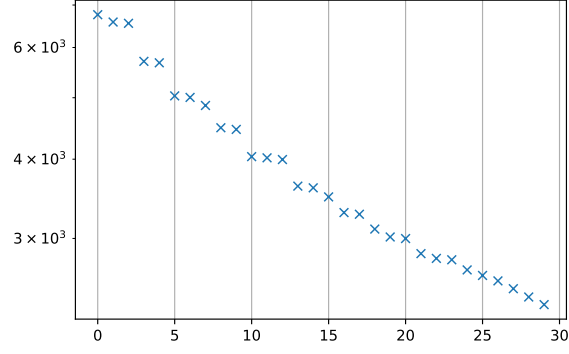
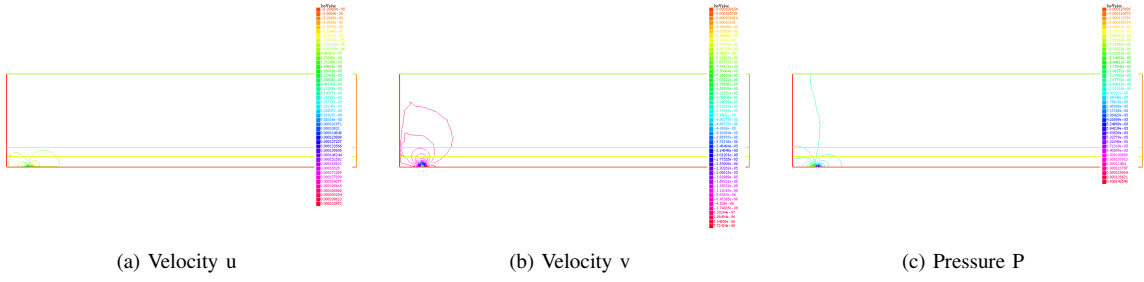
Figure 5: POD Modes for $p = 30$ 

Figure 6: Contours of the first mode

C. Galerkin Projection

We recall the equation for perturbation dynamics :

$$\mathcal{B}\dot{w}'(t) = -\mathcal{L}_{w_0}w'(t) + \mathcal{B}w_fc(t)$$

Or in semi-discretized form :

$$Q\dot{\hat{X}} = A\hat{X} + QCc$$

Given POD basis $XV = \hat{X}$ yields :

$$\hat{\dot{X}} = \underbrace{V^*AV}_{\hat{A} \in \mathbb{R}^{p \times p}} \hat{X} + \underbrace{V^*QC}_{\hat{C}} c$$

To determine the forced response of the dynamical system, we consider a periodic control law in the form of $c(t) = \cos(\omega t)$, where ω is a real frequency. For the ROM performed in figure , the frequency is set at $\omega = 10$.

The time extent is restricted due to a singularity that occurs after this specific interval. Thus the fit between ROM and DNS is rather acceptable at the beginning of simulations than diverges as the iterative process exceeds 1000 iterations (150000) .

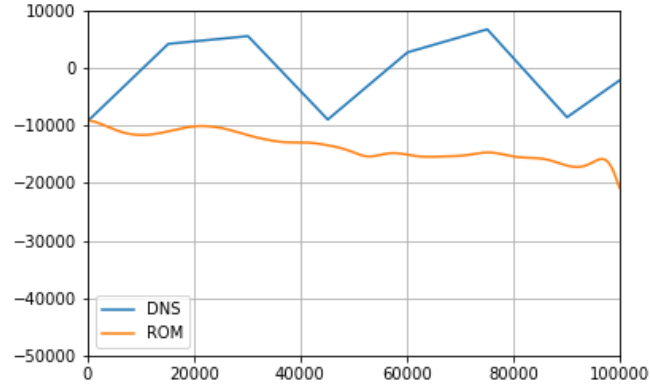


Figure 7: Simulation of ROM compared to DNS, for the second POD mode

III. CONCLUSION

In conclusion, in this project, we have established the theory of a pulse on a flow and we have realized a simulation. However, this type of DNS simulation is very resource-intensive. Therefore, we implemented singular value decomposition methods to extract the most important modes of the system. The POD method was used to identify the most dominant modes of the dynamics in our data set. The POD method requires a large amount of data, which made it necessary to reduce the model in order to reduce the complexity of the system, make it easier to analyze, and reduce computational time. Our project also involves the use of an expansion basis and a Petrov-Galerkin projection to construct the reduced system. Using the POD method, the expansion basis is made up of the most important modes identified by the method and the Petrov-Galerkin projection must then be used to project the original system onto the expansion basis to construct the reduced system. However, the results we have obtained need to be tested and verified. In summary, this project uses the mathematical techniques developed in our "Shear Flow Instability Control" course to reduce the complexity of dynamical systems while maintaining the key characteristics of the original system. We apply them here to fluid mechanics; however POD, SVD, ROM... are widely used in various applications in engineering, applied sciences and for processing large databases.

APPENDIX A

```

1  include "../Common/intro.edp"
2
3
4  Uv select=(x<440000.)*(x<460000.);
5  macro measure(u) int1d(th,2)(dy(u)*select) //
6
7  real dt=150.;
8  int nbre=6000;
9  int step=10;
10
11  int tps=0;
12  Uvvp [u1,u2,p],[sm1,sm2,sm3];
13  Uvvp [up1,up2,pp];
14  Uvvp [upp1,upp2,ppp];
15
16
17  real a0,a1,a2,bet0,bet1;
18  a0=1./dt; a1=-1./dt; a2=0; bet0=1; bet1=0; // coefficients for first-order in time integration
19
20  Uvvp [ub1,ub2,pb];
21  {
22      ifstream file("../BF/cbf.txt");
23      file >> ub1[];
24  };
25
26  // Check CFL number and mesh Reynolds-number
27  {
28      Uv cfl=sqrt(ub1^2+ub2^2)*dt/hTriangle;
29      Uv rem=sqrt(ub1^2+ub2^2)*hTriangle/nu;
30      cout << "cfl max=" << cfl[] << endl;
31      cout << "rem max=" << rem[] << endl;
32      cout << "u2[]" << u2[] << endl;
33      plot(cfl,wait=1,value=1,fill=1);
34      plot(rem,wait=1,value=1,fill=1);
35  }
36
37  varf LNS(du1,du2,dp],[v1,v2,q])=
38  int2d(th){
39      a0*du1*v1+a0*du2*v2
40      +nu*(dx(du1)*dx(v1)+dy(du1)*dy(v1)+dx(du2)*dx(v2)+dy(du2)*dy(v2))
41      +v1*(du1*dx(ub1)+du2*dy(ub1)+ub1*dx(du1)+ub2*dy(du1))
42      +v2*(du1*dx(ub2)+du2*dy(ub2)+ub1*dx(du2)+ub2*dy(du2))
43      -dp*(dx(v1)+dy(v2))
44      +q*(-dx(du1)-dy(du2))
45      +on(1,2,du1=0,du2=0)
46      +on(6,du2=0);
47
48  varf Mass(du1,du2,dp],[v1,v2,q])
49      =int2d(th)(du1*v1+du2*v2);
50
51  matrix MatLNS=LNS(Uvvp,Uvvp,solver=sparsecg);
52  matrix MatPscal=Mass(Uvvp,Uvvp,solver=CG);
53
54
55
56  varf bc[du1,du2,dp],[v1,v2,q]=on(2,du1=0,du2=(x<=58000)*(x<=62000));
57  Uvvp [f1,f2,f3]; // Weak form of forcing projected in stable subspace
58  f1[]=bc(0,Uvvp);
59
60  real u=1; // At initial time, control is equal to 1
61
62  u1[]=0; // At initial time, flowfield is equal to zero
63  up1[]=0;
64
65  string filename="cavite.txt"; // output file at each time step
66  {
67      ofstream f(filename);
68      f.precision(16);
69      // cout << " " << measure(u1) << " " << u << endl;
70  }
71  {
72      ofstream f(filename2);
73      f.precision(16);
74      f << tps << " " << measure(u1) << endl;
75  };
76
77  for (int i=1; i<=nbre; i++)
78  {
79      upp1[]=up1[];
80      up1[]=MatPscal*u1[];
81
82      if(i==2)
83      {
84          a0=1.5/dt; a1=-2./dt; a2=0.5/dt; bet0=2; bet1=-1;
85          MatLNS=LNS(Uvvp,Uvvp,solver=sparsecg);
86      }
87
88      sm1[]=-a1*up1[]-a2*upp1[]; // build right-hand-side
89      sm1[]+=u*f1[]; // take into account forcing
90      u1[]=MatLNS^-1*sm1[]; // compute new flowfield
91      tps+=dt; // advance time-step
92      if(i<step) // New control
93      {
94          u=1;
95      }
96      else
97      {
98          u=0;
99      }
100      if(i%(step)) // outputs at each time step
101      {
102          cout << " i = " << i << endl;
103          cout << " tps = " << tps << endl;
104      }
105      {
106          ofstream f(filename,append);
107          f.precision(16);
108          f << tps << " " << measure(u1) << " " << u << endl;
109      };
110
111      if((i%step)==0) // outputs at sampling time
112      {
113          ofstream f(filename2,append);
114          f.precision(16);
115          f << tps << " " << measure(u1) << endl;
116      };
117
118      sm1[]=MatPscal*u1[];
119      real nrip=u1[]*sm1[];
120
121      // Write output
122      {
123          ofstream filephi(filename2,append);
124          filephi << tps << " "
125          << nrip << " "
126          << u1(1,.0) << " "
127          << u2(1,.0) << " "
128          << u1(5,.0) << " "
129          << u2(5,.0) << endl;
130      };
131      // Save current flowfield every 1000 time steps
132      if((i%step)==0)
133      {
134          ofstream file("vect/utot_"+i+".txt");
135          file << u1[] << endl;
136          ofstream file2("iter.txt");
137          file2 << i << endl;
138      };
139  };
140

```

Figure 8: Code to generate of the impulse dns

```

1  #include "../Common/intro.edp"
2
3  {
4      //
5      // Linearized Navier-Stokes matrix
6      //
7
8      // Molecular kinematic viscosity
9      real nu=1.;
10     // Read base-flow (the code is written in perturbative form)
11     Uvvp [ub1,ub2,pb1];
12     {
13         ifstream file("../BF/cbf.txt");
14         file >> ub1[];
15     }
16     // Linearized Navier-Stokes operator with time-stepping
17     varf NS([du1,du2,dp],[v1,v2,q])=
18         int2d(th)(
19             -nu*(dx(du1)*dx(v1)+dy(du1)*dy(v1)
20                 +dx(du2)*dx(v2)+dy(du2)*dy(v2))
21             - v1*(du1*dx(ub1)+du2*dy(ub1)+ub1*dx(du1)+ub2*dy(du1))
22             - v2*(du1*dx(ub2)+du2*dy(ub2)+ub1*dx(du2)+ub2*dy(du2))
23             + dp*(dx(v1)+dy(v2))
24             + q*(dx(du1)+dy(du2)));
25
26     matrix A=NS(Uvvp,Uvvp,solver=GMRES);
27     {
28         ofstream file("A.txt");
29         file.precision(16);
30         file << A << endl;
31     }
32 }
33
34 {
35     //
36     // derivative and mass matrices
37     //
38
39     varf derx([a,b,c],[v1,v2,q])
40     = int2d(th)(v1*dx(a)+v2*dx(b));
41     matrix MatDerx=derx(Uvvp,Uvvp,solver=GMRES);
42
43     varf dery([a,b,c],[v1,v2,q])
44     = int2d(th)(v1*dy(a)+v2*dy(b));
45     matrix MatDery=dery(Uvvp,Uvvp,solver=GMRES);
46
47     varf mass([u1,u2,p],[v1,v2,q]) // mass matrix
48     = int2d(th)(u1*v1+u2*v2);
49
50     // derivative and mass matrices
51     //
52
53     varf derx([a,b,c],[v1,v2,q])
54     = int2d(th)(v1*dx(a)+v2*dx(b));
55     matrix MatDerx=derx(Uvvp,Uvvp,solver=GMRES);
56
57     varf dery([a,b,c],[v1,v2,q])
58     = int2d(th)(v1*dy(a)+v2*dy(b));
59     matrix MatDery=dery(Uvvp,Uvvp,solver=GMRES);
60
61     varf mass([u1,u2,p],[v1,v2,q]) // mass matrix
62     = int2d(th)(u1*v1+u2*v2);
63     matrix MassMat=mass(Uvvp,Uvvp,solver=GMRES);
64
65     {
66         ofstream file("M.txt");
67         file.precision(16);
68         file << MassMat << endl;
69     }
70
71     {
72         ofstream file("DerX.txt");
73         file.precision(16);
74         file << MatDerx << endl;
75     }
76
77     {
78         ofstream file("DerY.txt");
79         file.precision(16);
80         file << MatDery << endl;
81     }
82 }
83
84 {
85     //
86     // Structure of (u,v,p) vector : u=1, v=2, p=3
87     //
88
89     Uvvp [u1,u2,p];
90     [u1,u2,p]=[1,2,3];
91     {
92         ofstream file("StructVect.txt");
93         file << u1[] << endl;
94     }
95 }

```

Figure 9: Code to export the Matrix from the dns simulation

```

1  # -*- coding: utf-8 -*-
2  """
3  Created on Thu Jun 21 17:19:51 2018
4
5  @author: sipp
6  """
7
8  import numpy as np
9  import funcIO
10
11  mass=funcIO.ReadMatrix('M')
12  state=funcIO.ReadSnap('Snap')
13
14  print('Generate POD modes')
15
16  p = 30 # number of POD modes to compute
17
18  nbre=state.shape[1] # number of snapshots
19  state=state[:,1:6000] # restrict state to selected snapshots
20
21  C=state.transpose()@mass@state
22  # use state and mass matrices to compute POD modes
23  t , mu , vh =np.linalg.svd(C, full_matrices=True)
24  tp=t[:,0:p]
25  s=mu[0:p]**0.5 # singular values
26  v = state@tp@np.diag(s**-1)# POD modes
27
28
29  np.savez('PODmodes.npz', v=v, s=s) # save POD modes in python binary format
30
31  print('Write POD modes in POD folder')
32
33  for j in range(1,p+1):
34      print('Writing POD mode n°',j,'/',p)
35      np.savetxt('POD/mode_'+str(j)+'_'.txt',v[:,j-1], delimiter='\n', header=str(v.shape[0]), comments='')
36
37  for j in range(1,p+1):
38      print('Writing chronos mode n°',j,'/',p)
39      np.savetxt('POD/chronos_'+str(j)+'_'.txt',t[:,j-1], delimiter='\n')
40

```

Figure 10: Code to generate the POD

```

7
8  print('Compute ROM')
9
10 import numpy as np
11 import matplotlib.pyplot as plt
12 import funcIO
13
14 mass = funcIO.ReadMatrix('M') # load mass matrix
15 lns = funcIO.ReadMatrix('A') # load mass matrix
16 state = funcIO.ReadSnap('Snap')
17 POD = np.load('PODmodes.npz')
18 v = POD['v'] # POD modes
19 s = POD['s'] # singular values
20 # build reduced matrices
21 Q=v.transpose().dot(mass.dot(v)) # should be identity
22 A=v.transpose().dot(lns.dot(v)) # reduced dynamics on selected POD modes
23 # Check eigenvalues of reduced matrix
24 ev=np.linalg.eigvals(A)
25 plt.plot(ev.real,ev.imag,'bo')
26 plt.grid()
27 plt.savefig('ROM.pdf')
28 plt.show()
29 print('Preprocessing')
30 # x and y derivative of POD modes
31 ddx = funcIO.ReadMatrix('DerX')
32 ddy = funcIO.ReadMatrix('DerY')
33 dvdx = ddx * v # The derivatives of the POD modes. ddx is a sparse matrix. Therefore * refers to matrix product
34 dvdy = ddy * v # The derivatives of the POD modes.
35 # generate vectors uu=(-u,-u,0)* and vv=(-v,-v,0)* where (u,v,p) is a POD mode
36 svect = funcIO.ReadStruct()
37 uu = np.zeros(v.shape, dtype=float)
38 vv = np.zeros(v.shape, dtype=float)
39 for ii in range(0,svect.size):
40     if svect[ii]==1: # u component
41         uu[ii,:]=-v[ii,:] # u component
42         uu[ii+1,:]=-v[ii,:] # u component
43         vv[ii,:]=-v[ii+1,:] # after a u component there is always a v component
44         vv[ii+1,:]=-v[ii+1,:] # after a u component there is always a v component
45     ...
46 print('Generate cubic nonlinearities')
47 N = np.zeros((v.shape[1],v.shape[1],v.shape[1]), dtype=float)
48 for j in range(0,v.shape[1]):
49     print('N',j,'l,m')
50     for k in range(0,v.shape[1]):
51         temp = uu[:,j]*dvdx[:,k]+vv[:,j]*dvdy[:,k] # with arrays, * refers to component-wise product
52         for i in range(0,v.shape[1]):
53             ww=v[:,i]*temp
54             N[j,k,i]=np.sum(ww)
55     ...
56 print('save ROM in binary format')
57 np.savez('ROM.npz', A=A)

```

Figure 11: Code to generate the ROM

REFERENCES

- [1] Barbagallo, Alexandre, Denis Sipp, and Peter J. Schmid. "Closed-loop control of an open cavity flow using reduced-order models." *Journal of Fluid Mechanics* 641 (2009): 1-50.
- [2] Barbagallo, Alexandre. *Model reduction and closed-loop control of oscillator and noise-amplifier flows*. Diss. Palaiseau, Ecole polytechnique, 2011.
- [3] Barkley, Dwight, M. Gabriela M. Gomes, and Ronald D. Henderson. "Three-dimensional instability in flow over a backward-facing step." *Journal of fluid mechanics* 473 (2002): 167-190.
- [4] Sipp, Denis, and Anton Lebedev. "Global stability of base and mean flows: a general approach and its applications to cylinder and open cavity flows." *Journal of Fluid Mechanics* 593 (2007): 333-358.